

Bare Demo of IEEEtran.cls for IEEE Conferences

Michael Shell
Georgia Institute of Technology
someemail@somedomain.com

Homer Simpson
Twentieth Century Fox
homer@thesimpsons.com

James Kirk
and Montgomery Scott
Starfleet Academy
someemail@somedomain.com

Abstract—The abstract goes here.

I. INTRODUCTION

This demo file is intended to serve as a “starter file” for IEEE conference papers produced under L^AT_EX using IEEEtran.cls version 1.8b and later. I wish you the best of success.
¹

mds

August 26, 2015

II. THE HISTORY OF THE NATIONAL HOCKEY LEAGUE

III. CONCLUSION

The conclusion goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.

Algorithm 1 OMBSUBSAMPLE(D, n, m, k)

Input: Dataset D of size n , sample size m , number of samples k

Output: Sample list $\mathcal{S}$

Offline phase

- 1: $M \leftarrow \text{MARKMATRIX}(n, m, k)$
 - 2: **if** $m \cdot k > n$ **then**
 - 3: $\mathcal{F} \leftarrow \text{FRONTIER}(0, k, n, m)$
 - 4: **else**
 - 5: $\mathcal{F} \leftarrow \perp$
 - 6: **end if**
 - 7: $C \leftarrow \text{CONTROLBITS}(M, \mathcal{F}, n, m, k)$
 - 8: **Online phase**
 - 9: $\mathcal{S} \leftarrow \text{RECSAMPLE}(D, C, \mathcal{F}, n, m, k)$
 - 9: **return** $\mathcal{S}$
-

Algorithm 2 MARKMATRIX(n, m, k)

Input: Dataset size n , sample size m , number of samples k
Output: A length- n array M , where each $M[i]$ is a k -bit membership word

- 1: $M \leftarrow$ length- n array filled with 0
 - 2: $R \leftarrow$ length- k array filled with m
 - 3: **for** $i \leftarrow 0 \dots n - 1$ **do**
 - 4: **for** $j \leftarrow 0 \dots k - 1$ **do**
 - 5: Generate $r \in [0, 1)$ uniformly at random
 - 6: $b \leftarrow \mathbf{1}\{r < R[j]/(n - i)\}$
 - 7: $M[i] \leftarrow M[i] \mid (b \ll j)$
 - 8: $R[j] \leftarrow R[j] - b$
 - 9: **end for**
 - 10: **end for**
 - 11: **return** M
-

¹text for footnote

Algorithm 3 CONTROLBITS($M, \mathcal{F}, n, m, k$)

Input: Membership-word array M , Frontier list $\mathcal{F}$, dataset size n , sample size m , number of samples k

Output: One-dimensional control array C

- 1: $L \leftarrow \text{CONTROLNUM}(\mathcal{F}, n, m, k)$
 - 2: Create a bit array C of length L
 - 3: $p \leftarrow \text{CONTROLWRITE}(M, C, \mathcal{F}, m, k, 0)$
 - 4: **assert** $p = L$
 - 5: **return** C
-

Algorithm 4 RECSAMPLE($D, C, \mathcal{F}, n, m, k$)

Input: Dataset D of size n , control array C , Frontier list $\mathcal{F}$, sample size m , number of samples k

Output: A list $\mathcal{S} = [S_0, \dots, S_{k-1}]$ of k samples, each of size m

- 1: $(\mathcal{S}, p) \leftarrow \text{CONTROLREAD}(D, C, \mathcal{F}, m, k, 0)$
 - 2: **assert** $p = |C|$
 - 3: **return** $\mathcal{S}$
-

Algorithm 5 CONTROLNUM($\mathcal{F}, n, m, k$)

Input: Frontier list $\mathcal{F}$, Current input length n , Sample size m , number of samples k

Output: Number of control bits L

- 1: **if** $k = 1$ **then**
 - 2: **return** 0
 - 3: **end if**
 - 4: $\mathcal{V} \leftarrow \text{NEXTNODES}(\mathcal{F}, k)$
 - 5: $L \leftarrow 0$
 - 6: **for all** $(s, \ell) \in \mathcal{V}$ **do**
 - 7: $t_v \leftarrow \min(n, m \cdot \ell)$
 - 8: $L \leftarrow L + n$
 - 9: $L \leftarrow L + \text{CONTROLNUM}(\perp, t_v, m, \ell)$
 - 10: **end for**
 - 11: **return** L
-

Algorithm 6 NEXTNODES($\mathcal{F}, k$)

Input: Frontier list $\mathcal{F}$, number of samples k

Output: A list $\mathcal{V}$ of nodes (s, ℓ)

- 1: **if** $\mathcal{F} \neq \perp$ **then**
 - 2: **return** $\mathcal{F}$
 - 3: **end if**
 - 4: $k_l \leftarrow \lfloor k/2 \rfloor$
 - 5: $k_r \leftarrow k - k_l$
 - 6: **return** $[(0, k_l), (k_l, k_r)]$
-

Algorithm 7 CONTROLWRITE($M, C, \mathcal{F}, m, k, p$)

Input: Current membership-word array M , Global control array C , Frontier list $\mathcal{F}$, Sample size m , number of samples k , Writing position p

Output: Next free position p

- 1: $n \leftarrow |M|$
 - 2: **if** $k = 1$ **then**
 - 3: **return** p
 - 4: **end if**
 - 5: $\mathcal{V} \leftarrow \text{NEXTNODES}(\mathcal{F}, k)$
 - 6: **for** $i \leftarrow 0 \dots |\mathcal{V}| - 1$ **do**
 - 7: $(s, \ell) \leftarrow \mathcal{V}[i]$
 - 8: $t_v \leftarrow \min(n, m \cdot \ell)$
 - 9: $\text{mask}_v \leftarrow (1 \ll \ell) - 1$
 - 10: Create an n -bit vector C_v
 - 11: **for** $j \leftarrow 0$ **to** $n - 1$ **do**
 - 12: $C_v[j] \leftarrow 1\{((M[j] \gg s) \& \text{mask}_v) \neq 0\}$
 - 13: **end for**
 - 14: $C[p : p + n] \leftarrow C_v$
 - 15: $p \leftarrow p + n$
 - 16: $M_v \leftarrow \text{FILTERMARK}(M, C_v, t_v, s, \ell)$
 - 17: $p \leftarrow \text{CONTROLWRITE}(M_v, C, \perp, m, \ell, p)$
 - 18: **end for**
 - 19: **return** p
-

Algorithm 8 CONTROLREAD($D, C, \mathcal{F}, m, k, p$)

Input: Current dataset D , Global control array C , Frontier list $\mathcal{F}$, Sample size m , number of samples k , Reading position p

Output: Sample list $\mathcal{S}$ generated by the current node and next unread position p

- 1: $n \leftarrow |D|$
 - 2: **if** $k = 1$ **then**
 - 3: $S \leftarrow D[0 : m]$
 - 4: $\text{OSHUFFLE}(S)$
 - 5: **return** $([S], p)$
 - 6: **end if**
 - 7: $\mathcal{V} \leftarrow \text{NEXTNODES}(\mathcal{F}, k)$
 - 8: $\mathcal{S} \leftarrow []$
 - 9: **for** $i \leftarrow 0 \dots |\mathcal{V}| - 1$ **do**
 - 10: $(s, \ell) \leftarrow \mathcal{V}[i]$
 - 11: $t_v \leftarrow \min(n, m \cdot \ell)$
 - 12: $C_v \leftarrow C[p : p + n]$
 - 13: $p \leftarrow p + n$
 - 14: $D_v \leftarrow \text{FILTERDATA}(D, C_v, t_v)$
 - 15: $(\mathcal{S}_v, p) \leftarrow \text{CONTROLREAD}(D_v, C, \perp, m, \ell, p)$
 - 16: $\mathcal{S} \leftarrow \mathcal{S} \parallel \mathcal{S}_v$
 - 17: **end for**
 - 18: **return** $(\mathcal{S}, p)$
-

Algorithm 9 FRONTIER(s, ℓ, n, m)

Input: Sample start index s , Number of samples ℓ , Current input length n , Sample size m

Output: A list $\mathcal{F}$ of frontier nodes

```
1: if  $\ell = 1$  or  $m \cdot \ell < n$  then
2:   return  $[(s, \ell)]$ 
3: end if
4:  $\ell_l \leftarrow \lfloor \ell/2 \rfloor$ ,  $\ell_r \leftarrow \ell - \ell_l$ 
5:  $\mathcal{F}_l \leftarrow \text{FRONTIER}(s, \ell_l, n, m)$ 
6:  $\mathcal{F}_r \leftarrow \text{FRONTIER}(s + \ell_l, \ell_r, n, m)$ 
7: return  $\mathcal{F}_l \parallel \mathcal{F}_r$ 
```

Algorithm 10 FILTERDATA(D_v, C_v, t)

Input: Data array D_v of length n_v , Control vector $C_v \in \{0, 1\}^{n_v}$, Public output length t

Output: Filtered data array D' of length t

```
1: Copy  $D_v$  and store it in  $D'$ 
2: OCOMPACT( $D', C_v$ )
3: return  $D'[0 : t]$ 
```

Algorithm 11 FILTERMARK(M_v, C_v, t, s, ℓ)

Input: Membership-word array M_v of length n_v , Control vector $C_v \in \{0, 1\}^{n_v}$, Public output length t , Starting bit position s , Number of bits to keep ℓ

Output: Filtered membership-word array M' of length t

```
1: Copy  $M_v$  and store it in  $M_{\text{tmp}}$ 
2: OCOMPACT( $M_{\text{tmp}}, C_v$ )
3:  $\text{mask} \leftarrow (1 \ll \ell) - 1$ 
4: Create a length- $t$  array  $M'$ 
5: for  $i \leftarrow 0 \dots t - 1$  do
6:    $M'[i] \leftarrow (M_{\text{tmp}}[i] \gg s) \& \text{mask}$ 
7: end for
8: return  $M'$ 
```

Algorithm 12 CONTROLLAYOUT($\mathcal{V}, n, m, \omega$)

Input: Ordered node list $\mathcal{V}$, input length n , sample size m , base offset ω

Output: Ordered layout list $\mathcal{P}$

```
1:  $\mathcal{P} \leftarrow []$ 
2:  $\alpha \leftarrow \omega$ 
3: for  $j \leftarrow 0 \dots |\mathcal{V}| - 1$  do
4:    $(s, \ell) \leftarrow \mathcal{V}[j]$ 
5:    $t_v \leftarrow \min(n, m \cdot \ell)$ 
6:    $L_v \leftarrow \text{CONTROLNUM}(\perp, t_v, m, \ell)$ 
7:    $\mathcal{P} \leftarrow \mathcal{P} \parallel [(s, \ell, t_v, \alpha, \alpha + n)]$ 
8:    $\alpha \leftarrow \alpha + n + L_v$ 
9: end for
10: return  $\mathcal{P}$ 
```

Algorithm 13 CONTROLWRITEPAR($M, C, \mathcal{F}, m, k, \omega$)

Input: Current membership-word array M , Global control array C , Frontier list $\mathcal{F}$, Sample size m , number of samples k , Base writing offset ω

```
1:  $n \leftarrow |M|$ 
2: if  $k = 1$  then
3:   return
4: end if
5:  $\mathcal{V} \leftarrow \text{NEXTNODES}(\mathcal{F}, k)$ 
6:  $\mathcal{P} \leftarrow \text{CONTROLLAYOUT}(\mathcal{V}, n, m, \omega)$ 
7: parallel for  $j \leftarrow 0 \dots |\mathcal{P}| - 1$ 
8:    $(s, \ell, t_v, \omega_C, \omega_{\text{sub}}) \leftarrow \mathcal{P}[j]$ 
9:    $\text{mask}_v \leftarrow (1 \ll \ell) - 1$ 
10:  Create an  $n$ -bit vector  $C_v$ 
11:  for  $i \leftarrow 0$  to  $n - 1$  do
12:     $C_v[i] \leftarrow 1\{((M[i] \gg s) \& \text{mask}_v) \neq 0\}$ 
13:  end for
14:   $C[\omega_C : \omega_C + n] \leftarrow C_v$ 
15:   $M_v \leftarrow \text{FILTERMARK}(M, C_v, t_v, s, \ell)$ 
16:   $\text{CONTROLWRITEPAR}(M_v, C, \perp, m, \ell, \omega_{\text{sub}})$ 
17: end parallel for
18: return
```

Algorithm 14 CONTROLREADPAR($D, C, \mathcal{F}, m, k, \omega$)

Input: Current dataset D , Global control array C , Frontier list $\mathcal{F}$, Sample size m , number of samples k , Base reading offset ω

Output: Sample list $\mathcal{S}$ generated by the current node

```
1:  $n \leftarrow |D|$ 
2: if  $k = 1$  then
3:    $\mathcal{S} \leftarrow D[0 : m]$ 
4:   OSHUFFLE( $\mathcal{S}$ )
5:   return  $[\mathcal{S}]$ 
6: end if
7:  $\mathcal{V} \leftarrow \text{NEXTNODES}(\mathcal{F}, k)$ 
8:  $\mathcal{P} \leftarrow \text{CONTROLLAYOUT}(\mathcal{V}, n, m, \omega)$ 
9: parallel for  $j \leftarrow 0 \dots |\mathcal{P}| - 1$ 
10:   $(s, \ell, t_v, \omega_C, \omega_{\text{sub}}) \leftarrow \mathcal{P}[j]$ 
11:   $C_v \leftarrow C[\omega_C : \omega_C + n]$ 
12:   $D_v \leftarrow \text{FILTERDATA}(D, C_v, t_v)$ 
13:   $\mathcal{S}_j \leftarrow \text{CONTROLREADPAR}(D_v, C, \perp, m, \ell, \omega_{\text{sub}})$ 
14: end parallel for
15: return  $\bigcup_{j=0}^{|\mathcal{P}|-1} \mathcal{S}_j$ 
```

Algorithm 15 OMBSUBSAMPLEPAR(D, n, m, k)

Input: Dataset D of size n , sample size m , number of samples k

Output: Sample list $\mathcal{S}$

Offline phase

- 1: $M \leftarrow \text{MARKMATRIX}(n, m, k)$
- 2: **if** $m \cdot k > n$ **then**
- 3: $\mathcal{F} \leftarrow \text{FRONTIER}(0, k, n, m)$
- 4: **else**
- 5: $\mathcal{F} \leftarrow \perp$
- 6: **end if**
- 7: $L \leftarrow \text{CONTROLNUM}(\mathcal{F}, n, m, k)$
- 8: Create a global control array C of length L
- 9: $\text{CONTROLWRITEPAR}(M, C, \mathcal{F}, m, k, 0)$

Online phase

- 10: $\mathcal{S} \leftarrow \text{CONTROLREADPAR}(D, C, \mathcal{F}, m, k, 0)$
 - 11: **return** $\mathcal{S}$
-